

Lab 4: VibeCoding**Objective**

Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account.

THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!

Duration

60 minutes

Software Requirements

- Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)

Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10. Do not raise PR to the main repo (SETAPESU26)

Deliverables:

Upload the following to your Lab repo under Lab-4 folder,

- a. Videos (before and after)
- b. Updated Code
- c. Chat history exported as a doc/pdf

 [satvik-J/50_traffic-escape](#)

tell me step by step what to do

👍 I went through the assignment instructions and your actual GitHub repo. Your fork is satvik-J/50_traffic-escape , forked from the assigned SETAPESU26/50_traffic-escape . The repo currently contains main.py , requirements.txt , the game/ folder, and a README with the 4 features you are required to add.  GitHub +1 

What you have to do

Your assignment is basically:

Clone → run original → record 10 sec → use Vibe Coding/AI → fix/add 4 features → test → record 10 sec → push → upload videos + updated code + chat history.

The four required features in your README are:  GitHub

1. ❤️ Lives System
2. 🚣 Moving Log/Raft Lane
3. 🏆 Top 5 High Score Table
4. 🌙 Day/Night Cycle

STEP 1 — Clone your repo

Open VS Code.

Open Terminal:

```
git clone https://github.com/satvik-1/50_traffic-escape.git
```

Bash

Then:

Bash

You should see something like:

```
50_traffic-escape/
|
├── game/
|   ├── __init__.py
|   ├── game_engine.py
|   ├── player.py
|   └── traffic.py
|
├── main.py
├── requirements.txt
└── README.md
```

That matches the structure specified by the repository.  GitHub

STEP 2 — Install dependencies

Inside the VS Code terminal:

Bash

If `pip` doesn't work:

Bash

STEP 3 — Run the ORIGINAL game

Before touching **ANY** code, run:

Bash

The README says this is the normal way to launch the game.  [GitHub](#)

Test the controls:

Key	Action
W / ↑	Move forward
S / ↓	Move backward
A / ←	Change lane
D / →	Change lane
R	Restart

These are the existing controls in your repo.  [GitHub](#)

STEP 4 — Record the BEFORE video

This is important.

Your assignment specifically wants:

A 10-second video before your changes, showing the bug/broken behavior.

So do not modify anything before this recording.

Use Windows screen recording:

Option 1 — Xbox Game Bar

Press:

`Win + G`

Then start recording.

Or:

`Win + Alt + R`

Start the game and record around **10 seconds**.

What should you record?

Show the original game running.

Ideally demonstrate that the current version **doesn't have the required features**.

For example:

- get hit by a car
- show that there isn't a 3-life system
- play normally
- show the existing gameplay

Don't worry about making this video fancy.

Name it:

`before.mp4`

STEP 5 — NOW use your Vibe Coding tool

This is the actual point of the lab.

You can use **ChatGPT, Gemini, Claude, Cursor, etc.** The assignment specifically says to use a Vibe Coding tool. Your goal is to give prompts and have the AI modify the code.

Since you're talking to me here, you can use **this chat** as your LLM chat history.

VERY IMPORTANT

Don't just say:

"Add all the features."

The assignment specifically says:

Write prompts and add all the features listed in the README one by one.

So do them **one at a time**.

STEP 6 — Prompt #1: Lives System

Give your Vibe Coding AI this prompt:

```
I am working on the Traffic Escape Python game in this repository.
```

```
Read the existing code carefully before making changes.
```

```
Implement Task 1 from the README: Lives System.
```

```
Requirements:
```

- The player should start with exactly 3 lives.
- Whenever the player gets hit by a car, reduce the lives by 1.
- Display the current number of lives clearly on the game screen.
- When lives reach 0, show a Game Over state.
- The player should not immediately lose multiple lives from a single collision.
- Allow the player to restart the game using the existing R key behavior.
- Preserve all existing gameplay and controls.
- Do not remove or break any existing functionality.
- Make the smallest clean changes necessary.
- After making the changes, explain which files were modified and how the implementati



Let the AI modify the code.

Then **run the game**:

Bash

Test:

Test 1

Get hit once.

You should see:

```
Lives: 2
```

Test 2

Get hit again:

```
Lives: 1
```

Test 3

Get hit again:

```
Game Over  
Lives: 0
```

If it works → move on.

STEP 7 — Prompt #2: Moving Log/Raft

Now give the AI:

```
Now implement Task 2 from the README.
```

```
Add a safe water lane where the player must use a moving log/raft to cross.
```

```
Requirements:
```

- Add a dedicated safe/water lane.
- Add one or more logs/rafts that move horizontally across this lane.
- The player should be able to stand on the moving log.
- When the player is standing on the log, the player should move along with it.
- If the player enters the water without being on a log, treat it as a failed crossing
- The log should continuously move and wrap/reappear when it leaves the screen.
- Keep the existing traffic lanes and controls working.
- Integrate this cleanly with the existing game architecture.
- Do not remove the lives system we just implemented.

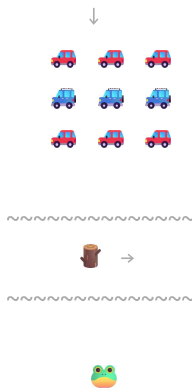
- Make only the necessary changes.
- Explain the files and logic you changed after implementing it.

Then:

Bash

Test:

Traffic lanes



You should be able to use the log to cross.

STEP 8 — Prompt #3: High Score Table

Then give the AI:

Now implement Task 3 from the README.

Add a persistent Top 5 High Score system.

Requirements:

- Track the player's score during gameplay.
- Display the current score on screen.
- When a game ends, allow the score to be considered for the leaderboard.
- Store the top 5 scores persistently in a JSON file.
- The scores must remain available after closing and reopening the game.
- Sort the scores from highest to lowest.
- Keep only the best 5 scores.
- Display the Top 5 leaderboard clearly in the game.
- If the JSON file does not exist, create it automatically.

- Handle invalid or missing JSON data safely.
- Do not break the existing lives, raft, traffic, controls, or game-over behavior.
- Make the implementation consistent with the existing project structure.

Run:

```
Bash
```

Play.

Get a score.

Close the game.

Run again:

```
Bash
```

Your high score should still exist.

You should end up with something similar to:

```
HIGH SCORES
```

```
1. 1250
2. 980
3. 750
4. 500
5. 320
```

And probably a file such as:

```
high_scores.json
```

in the project.

STEP 9 — Prompt #4: Day/Night Cycle

Finally:

Now implement Task 4 from the README.

Add a day/night cycle.

Requirements:

- The game should automatically switch between day and night every 30 seconds.
- The transition should continue throughout gameplay.
- Make the visual appearance clearly different between day and night.
- During night mode, car headlights should be visible and illuminate farther ahead.
- Headlights should be visually noticeable but should not interfere with collision det
- Display a small indicator showing whether the game is currently in DAY or NIGHT mode
- Keep all existing functionality working, including lives, moving raft, scoring, lead
- Use the existing game architecture rather than rewriting the entire project.
- Avoid unnecessary changes.
- Explain exactly how the 30-second timer and night headlights work.



Then run:

Bash

You need to test this one properly because waiting **30 seconds** is part of the requirement.

STEP 10 — Full testing

Now test everything together.

Run:

Bash

Check this list:

♥ Lives

3 → 2 → 1 → 0

Works?



Raft

Can you:

```
get onto log
      ↓
move with log
      ↓
cross water
```

Works?



Score

Does score increase?

Does leaderboard show?

Does it survive restarting the game?



Day/Night

Wait 30 seconds.

Does:

```
DAY
  ↓
NIGHT
```

happen?

Do headlights appear?

Wait another 30 seconds.

Does it switch back?



STEP 11 — Record the AFTER video

Now record another **10+ second video**.

The assignment wants the after video to show the fixed behavior and new features.

Don't just record the game sitting idle.

Try to capture multiple things.

For example:

Seconds 0–3

Show:

Score: 120

Lives: 3

DAY

Move around.

Seconds 3–6

Get hit.

Show:

Lives: 2

Seconds 6–10+

Show the raft/night mode/score/leaderboard if possible.

If you can't show everything in 10 seconds, make the video a little longer rather than hiding the functionality.

Name it:

after.mp4

STEP 12 — Check Git changes

In VS Code terminal:

Bash

You should see your modified files.

For example:

```
modified: game/game_engine.py
modified: game/player.py
modified: game/traffic.py
```

and possibly:

```
high_scores.json
```

depending on how the AI implemented it.

STEP 13 — Check the actual changes

Run:

Bash

Read this.

Make sure the AI didn't randomly rewrite your whole project.

You want to see changes related to:

```
Lives
Raft
High scores
Day/night
```

STEP 14 — Commit

Once you've tested everything:

Bash

Then:

Bash

Then:

Bash

Your GitHub repository should now contain the updated code.

Your repo currently has only one commit, so this will give you a clear record of your work.



STEP 15 — DO NOT create a PR

This is specifically written in your assignment:

Do not raise PR to the main repo (SETAPESU26)

So **DO NOT** do:

New Pull Request

You only need to push your work to:

satvik-J/50_traffic-escape

Your repo is already your fork of SETAPESU26/50_traffic-escape .  GitHub

STEP 16 — Prepare the Lab-4 submission

Your assignment says the Lab-4 folder should contain:

```
Lab-4/  
|  
├─ before.mp4  
├─ after.mp4  
├─ Updated Code  
└─ Chat History.pdf/docx
```

The repository's own README similarly specifies the before video, after video, and complete LLM chat history as the submission checklist.  GitHub

Your chat history

Since you're using ChatGPT, save/export this conversation as:

```
Chat_History.pdf
```

Make sure it includes the prompts you used and the AI's responses/code changes.

Your complete checklist

Before submitting, make sure you have:

- ☐ Cloned assigned repo
- ☐ Read README
- ☐ Installed requirements
- ☐ Successfully ran original game
- ☐ Recorded **BEFORE** video
- ☐ Used Vibe Coding tool
- ☐ Added 3 Lives
- ☐ Added Moving Log/Raft
- ☐ Added Top 5 Persistent High Scores
- ☐ Added 30-second Day/Night Cycle
- ☐ Added night headlights
- ☐ Tested everything
- ☐ Recorded **AFTER** video
- ☐ `git add .`

- ☐ git commit
- ☐ git push origin main
- ☐ Did **NOT** create PR to SETAPESU26
- ☐ Saved complete AI chat history
- ☐ Put required files under **Lab-4**

One important thing

Don't start modifying the code yet.

First do:

Bash

```
git clone https://github.com/satvik-J/50_traffic-escape.git
cd 50_traffic-escape
pip install -r requirements.txt
python main.py
```

Then **record the original game**.

After that, if you want, send me a screenshot of your VS Code/project after cloning, and I can take you through **Task 1 → Task 2 → Task 3 → Task 4 one at a time**, including exactly what to paste into the Vibe Coding tool and how to verify each change.



ChatGPT can make mistakes. Check important info.